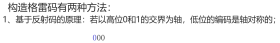
期末复习

- **大学英语**
- **问题求解**
- **数逻**
  - 题型1 选择
    - 信息编码
      - 定点数
        - 真值
        - 原码（符号位+单纯数值位）
          - +0、-0
          - 表示范围“少一个数”，正负对称
        - 补码（正数不变负数每位取反（含符号）+1）
          - +0、-0一样
          - 表示范围负数多一个
          - 为什么好?
            - PPT
        - 移码（偏移的补码）
        - 反码（逐位取反）
        - c语言应用
          - 机器数的解释
            - 用移码解释为二进制后比较
          - 精度丢失问题
            - 单精度，双精度与更小的int
        - 这数是多少？不确定！
      - 浮点数
        - 符号位、尾数和阶码（移码）
          - 阶码：0偏移至127，正数编码后-1
        - 规格化
          - 定义：小数点前只有一位非0数
          - 规格化的阶1码不全0也不全1
          - 全0：正负0、极小；全1：正负无穷（frac全0）、NaN
          - 尾码规格化
      - 非数值
        - BCD码
        - ASCII
          - 30-39,41,61（H）
        - 汉字
          - 区位码
          - 国标码=区位+2020
          - 机内码=国标+8080
        - 格雷码
          - 

            构造格雷码有两种方法：

            1、基于反射码的原理：若以高位0和1的交界为轴，低位的编码是轴对称的；

            000

00
01
11
10

001
011
010
110
111
101
100

2、利用二进制转换

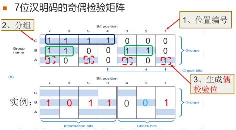
- N位二进制数字从右向左，从0到n-1编号；
- 如果第i位和第i+1位相同，则格雷码的第i位为0，否则为1（异或）。

  $G_n = B_n,\quad G_i = B_{i+1} \oplus B_i$

- 4位格雷码

Decimal	Binary	Gray	Decimal	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

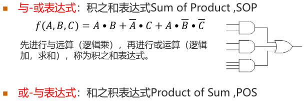
- 汉明码
  - 编码方法：
    - 从右向左给编码中的每个位置**编号**： 2^i-1 ~1；在2的幂次位置上**设置为校验位**，其余位置为信息位；
    - 如果信息位的位置编号和校验位的位置编号在相同的位置上值都是1，则这些信息位和校验位**组成一个组**。
      - 如校验位2(010)和信息位3(011)、6(110)、7(111)组成一组。
    - 每组生成1位偶校验位。
  - 7位汉明码的奇偶检验矩阵


2、分组
1、位置编号
Bit position
7
6
5
4
3
2
1
Group name
C
B
A
1 1 1 1 0 0 0
1 1 0 0 1 1 0
1 0 1 0 1 0 1
Groups
Check bits
3、生成偶校验位
(b)
Bit position
7
6
5
3
4
2
1
实例：
C
B
A
1 0 1 1 0 0 1
Groups
Information bits
Check bits

  - 1位错情况：j位=>非法编码。
    - 该错将引起包含j位的那些组的奇偶性不正确。
    - 每一位至少包含在一个组中。
  - 两位错：j和k
    - 同时包含j和k位的分组，奇偶性不变。
    - 因为j和k的二进制表示中，至少有1位不同，因此，j和k肯定有不相同的分组，从而导致对应的分组奇偶校验错误。
  - 增加全局奇偶校验，能更好判断
- 字
  - 小端和大端
    - 大端从最高有效字节MSB开始，从左到右直接读
    - 小端从最低有效字节LSB开始，从右到左合并读
  - 单位
    - 小b是0/1位，比特
    - 大B是字节，“白特”
    - KB...一次乘1024，KiB等等同意义，但带宽kb/s一次乘1000
- 布尔代数与组合电路
  - 真值表/假设法
  - 完备逻辑
    - 与非、或非
  - 对偶性
    - 积之和、和之积、最小项、最大项

  - 质蕴含项、实质蕴含项
    - 质蕴含项是卡诺图的“最大块”
    - 实质蕴含项是卡诺图“不可被替代的最大块”
    - 找完实质蕴含项后还要最小覆盖
  - 卡诺图
    - 00 01 11 10
  - 电路图与表达式一一对应，表达式和真值表不一一对应
- 认识电路&时序电路
  - 认识电路
    - 认识PMOS，NMOS
    - 电气特性
      - 噪声容积
      - 转换时间
      - 传输延迟
    - 三态门
  - 次态激励逻辑、状态记忆、输出逻辑、Mealy&Moore
  - 锁存与触发
    - 有效时改变，无效则锁住
    - 时钟每次变就触发
  - 状态与功能表达
    - 功能表
    - 有限状态图
    - 状态表
      - 状态转移表
        - 特征方程
    - 波形图
    - 状态编码
  - 双稳态元件的亚稳态
    - 脑子瓦特，俩反向输出一样
  - 时间
    - 建立时间
    - 锁存延迟
    - 保持时间
- 运算方法
  - 原码小数扩展
  - 补码左移与右移
  - 四位全先行进位加法器延迟6T
  - 补码加减的溢出判断用异或门实现
  - 做减法：全部取反+进位（不是补码！）
  - 做*2：右移位，正+正=负就是溢出
  - 正+正=负就是溢出
  - 正+负永不溢出
- 指令系统
  - 计算机层次
  - 指令：操作码、操作数（地址）
    - 指令集体系结构ISA

    - PC指令计数器，IR指令寄存器，PSWR指令状态寄存器
    - 寻址方式
      - 立即寻址
        - 操作数
      - 直接寻址
        - 给地址
      - 间接寻址
        - 间接一次
      - 寄存器寻址
        - 数据在寄存器这里
        - 寄存器间接
          - 地址在寄存器这里
      - 偏移寻址
        - 变址寻址
          - 变址寄存器+偏移
        - 相对寻址
          - PC+偏移
        - 基址寻址
          - 给定地址+偏移
    - RR指令，RS指令与SS指令
    - 中断、异常、故障、自陷、终止
    - 无条件转移，条件转移，过程调用（转子）
    - RISC、CISC
  - CPU
    - CPU/指令/程序的周期/频率
    - 指令与数据的访问时间不同
    - PC对用户程序不透明
    - PC存放后继指令地址，位数与驻村地址寄存器MAR相同
      - 主存数据寄存器MDR
    - 数据通路分析
    - “算个时间”
    - 微指令与微程序
    - 每个功能部件只能在每条指令中被使用一次
    - 分析冒险
      - 数据相关
      - 控制冒险
      - 转发？分口？插入几条nop
    - 静态动态预测
- 题型2 组合逻辑电路设计
  - 布尔代数化简
    -



■ 与-或表达式：积之和表达式Sum of Product ,SOP

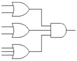
$$f(A,B,C) = A \bullet B + \overline{A} \bullet C + A \bullet \overline{B} \bullet \overline{C}$$

先进行与远算（逻辑乘），再进行或运算（逻辑加，求和），称为积之和表达式。

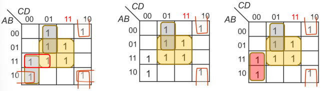
■ 或-与表达式：和之积表达式Product of Sum ,POS

$f(A,B,C) = (A+B)\bullet(\bar{A}+B+C)\bullet(A+\bar{C})$

先进行或远算（逻辑加，求和），再进行与运算（逻辑乘），称为和之积表达式。



- ■ $F(A,B,C,D) = \sum m(1,2,5,7,8,10,12,13,15)$


CD
AB
00
01
11
10
1

找出所有质蕴涵项

$B\cdot D, \bar{A}\cdot\bar{C}\cdot D, \bar{B}\cdot C\cdot\bar{D}, A\cdot B\cdot\bar{C}, A\cdot\bar{B}\cdot\bar{D}, A\cdot\bar{C}\cdot\bar{D}$

选择所有的实质蕴涵项

$B\cdot D+\bar{A}\cdot\bar{C}\cdot D+\bar{B}\cdot C\cdot\bar{D}$

选择最小的覆盖子集

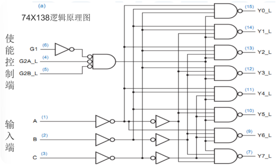
$A\cdot B\cdot\bar{C}, A\cdot\bar{B}\cdot\bar{D}, A\cdot\bar{C}\cdot\bar{D}$

$F(A,B,C,D)=B\cdot D+\bar{A}\cdot\bar{C}\cdot D+\bar{B}\cdot C\cdot\bar{D}+A\cdot\bar{C}\cdot\bar{D}$

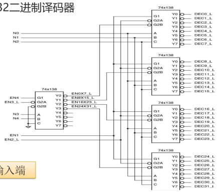
代数法化简结果：$\bar{A}\cdot\bar{C}\cdot D+\bar{B}\cdot C\cdot\bar{D}+B\cdot D+A\cdot\bar{B}\cdot\bar{D}+A\cdot B\cdot\bar{C}$

典型电路

n-2^n译码器


(a)
74X138逻辑原理图
使能控制端
G1 (6)
G2A_L (4)
G2B_L (5)
输入端
A (1)
B (2)
C (3)
(15) Y0_L
(14) Y1_L
(13) Y2_L
(12) Y3_L
(11) Y4_L
(10) Y5_L
(9) Y6_L
(7) Y7_L

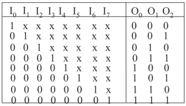
- 3-8译码器级联成5-32二进制译码器


74x138
N0
N1
N2
EN4
EN3_L
N3
N4
EN1
EN2_L
G1
G2A
G2B
A
B
C
Y0
Y1
Y2
Y3
Y4
Y5
Y6
Y7
EN0X7_L
EN8X15_L
EN16X23_L
EN24X31_L
DEC0_L
DEC7_L
DEC8_L
DEC15_L
DEC16_L
DEC23_L
DEC24_L
DEC31_L

最宽的门电路有4个输入端

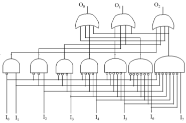
编码器

优先级编码器



I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	O_0	O_1	O_2
1	x	x	x	x	x	x	x	0	0	0
0	1	x	x	x	x	x	x	0	0	1
0	0	1	x	x	x	x	x	0	1	0
0	0	0	1	x	x	x	x	0	1	1
0	0	0	0	1	x	x	x	1	0	0
0	0	0	0	0	1	x	x	1	0	1
0	0	0	0	0	0	1	x	1	1	0
0	0	0	0	0	0	0	1	1	1	1

优先权编码器的功能描述

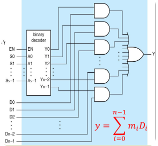

O_0
O_1
O_2
I_0
I_1
I_2
I_3
I_4
I_5
I_6
I_7

优先权编码器的逻辑电路图

- 加法器

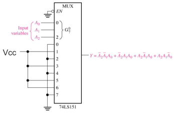
多路

多路选择


binary
decoder
EN
S0
S1
Ss−1
EN
A0
A1
As−1
Y0
Y1
Y2
Yn−2
Yn−1
D0
D1
D2
Dn−2
Dn−1
Y
$y = \sum_{i=0}^{n-1} m_i D_i$

利用八选一多路选择器实现:$f(A_2,A_1,A_0) = \sum m(1,3,5,6)$

m	A_2	A_1	A_0	D_i
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0


MUX
EN
Input variables
A_0
A_1
A_2
$G\frac{0}{7}$
Vcc
0
1
2
3
4
5
6
7
74LS151
$Y = \bar{A}_2\bar{A}_1A_0 + \bar{A}_2A_1A_0 + A_2\bar{A}_1A_0 + A_2A_1\bar{A}_0$

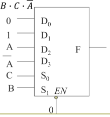
方法简单-查找表LOOKUP。
可利用数据输入线定义变量。

- 用4选1多路选择器实现F（A，B，C）=∑m(1,3,5,6)
  1. 选择B、C为选择变量，A为数据输入。
  2. 将函数表达式展开成选择变量最小项表示形式。
  3. 列出选择器的功能表，确定每条数据输入线的值。
  4. 画出逻辑电路图

$$f = \bar{A}\cdot\bar{B}\cdot C + \bar{A}\cdot B\cdot C + A\cdot\bar{B}\cdot C + A\cdot B\cdot\bar{C}$$
$$= \bar{B}\cdot\bar{C}\bullet 0 + \bar{B}\cdot C\cdot(A+\bar{A}) + B\cdot\bar{C}\cdot A + B\cdot C\cdot\bar{A}$$

选择 BC	数据输入
00	0
01	$A+\bar{A}=1$
10	A
11	$\bar{A}$

功能表

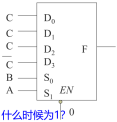

0
1
A
$\bar{A}$
C
B
D_0
D_1
D_2
D_3
S_0
S_1
EN
F
0

- F（A，B，C）=∑m(1,3,5,6)
  1. 选择AB为选择变量，C为数据输入；
  2. 已选择变量为表头画出卡诺图，标出实现函数的最小项；
  3. 确定每种选择情况下，数据输入线的值（0，1，原变量，反变量）。

$$F = \bar{A}\cdot\bar{B}\cdot C + \bar{A}\cdot B\cdot C + A\cdot\bar{B}\cdot C + A\cdot B\cdot\bar{C}$$

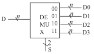
输入 C	选择 AB 00	01	11	10
0	0	2	6	4
1	1	3	7	5
D_i	C	C	$\bar{C}$	C

卡诺图实现表


C
C
C
$\bar{C}$
B
A
D_0
D_1
D_2
D_3
S_0
S_1
EN
F
0

思考：1什么情况下，数据输入线值为0，什么时候为1？
2在增加逻辑门电路控制的情况下，能否用更少的选择变量来实现布尔函数？

多路分配


D
n
DE
MU
X
00
01
10
11
D0
D1
D2
D3
2
S

四路分配器的符号

S0	S1	D0	D1	D2	D3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

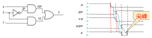
四路分配器真值表

- 桶形移位器

时序分析

冒险发现

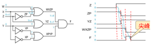
- 静态-1型冒险：期望输出保持稳定的1时，出现短暂的0。
- 分析：$F=X\cdot\bar{Z}+Y\cdot Z$，当XY=11，Z由1转变为0时的状态


X
Z
Y
ZP
XZP
YZ
F
Z
ZP
YZ
XZP
F
1
0
尖峰

主要存在于“与-或”SOP电路中：

输出函数在一定条件下，能简化成：$F = \overline{A\cdot\bar{A}} = A + \bar{A}$

- 静态-0型冒险：期望输出保持稳定的0时，出现短暂的1。
- 分析：当WXY=000，Z由0转变为1时的状态


W
X
Z
Y
ZP
YP
XP
WXZP
YZ
XPYP
F
Z
ZP
YZ
WXZP
F
尖峰

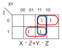
主要存在于“或一与”POS电路中：

输出函数在一定条件下，能简化成：F $=\overline{A+\bar{A}}=A\cdot\bar{A}$

$F=(W+X+\bar{Z})\cdot(Z+Y)\cdot(\bar{Y}+\bar{X})$

冒险消除

若卡诺图中，存在相切的卡诺圈，则可能出现静态冒险。


XY
Z
00 01 11 10
0
1
1 1
1 1

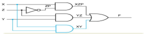
$X\cdot\bar{Z}+Y\cdot Z$

在什么条件下，出现静态冒险？

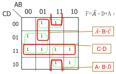
当X=1，Y=1时，出现$\bar{Z}+Z$

$F = X\cdot\bar{Z} + Y\cdot Z + X\cdot Y$

消除冒险：添加一致项X·Y，覆盖相切的乘积项。


X
Z
Y
ZP
XZP
YZ
XY
F

- 冒险发现与消除


AB
CD
00 01 11 10
00
01
11
10
1
$\bar{A}\cdot B\cdot\bar{C}$
C·D
$A\cdot B\cdot\bar{D}$

$F=\bar{A}\cdot D+A\cdot C+B\cdot\bar{C}\cdot\bar{D}$

添加冗余项：$\bar{A}\cdot B\cdot\bar{C}+C\cdot D+A\cdot B\cdot\bar{D}$

通过判断表达式也能发现，存在：$\bar{A}+A, C+\bar{C}, D+\bar{D}$

题型3 时序逻辑电路设计

稳态元件

- 串联反相器

SR锁存器-或非

- SFRF锁存器

D锁存器

- 使能控制

D触发器

- 引入使能控制!

- T触发器

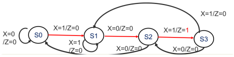
设计流程

1需求分析、构建状态图、状态表

- **构建状态图：**最困难的一步，相当于数学建模。
  - ① **首先**设定任一状态为电路初始状态；
  - ② **然后**从初始状态开始，分析每一个状态在不同输入组合作用下的状态转移和输出信息；
  - ③ 如果当某个状态下出现的输出响应（次态、输出）不能用已有状态表示时，则产生**新的状态**；
  - ④ 重复2、3两步，直到不产生新状态。

S0：假设为初始状态，等待接收输入　S2：接收到序列中两位有效数字10

S1：接收到序列中第一位有效数字1　S3：接收到序列中三位有效数字101


X=0
/Z=0
S0
X=1/Z=0
S1
X=1
/Z=0
X=0/Z=0
S2
X=1/Z=1
S3
X=1/Z=0
X=0/Z=0
X=0/Z=0

- 构建状态表
  - ◆ 用X表示输入数据；用Z表示检测结果；
  - ◆ 用S表示当前状态，S*表示次态。

状态表

	状态S	S*/Z	
		X=0	X=1
假设为初始状态，等待接收输入：	S0	S0/0	S1/0
接收到序列中第一位有效数字1：	S1	S2/0	S1/0
接收到序列中两位有效数字10：	S2	S0/0	S3/1
接收到序列中三位有效数字101：	S3	S2/0	S1/0

- 相辅相成
- 不让状态化简就别动，化简就是相同行合并

3状态编码、状态转移表

- 通常在具体设计时采用相邻法寻求次优编码方案：
  - ◆ 准则1：若两个现态的次态相同，则其对应编码应尽量相邻。
  - ◆ 准则2：同一个现态的各个次态其编码应尽量相邻。
  - ◆ 准则3：若两个现态的系统输出相同，则它们的编码应尽量相邻。

状态S	S*/Z	
	X=0	X=1
S0	S0/0	S1/0
S1	S2/0	S1/0
S2	S0/0	S1/1

- 根据准则1：S0和S1、S0和S2可相邻
- 根据准则2：S0和S1、S1和S2
- 根据准则3：S0、S1、S2可相邻
- 通常根据不同取舍得到不同编码方案。
- 若S0S1、S1S2相邻，则一种可能的编码方案如下：S0:00，S1:01，S2:11

- 状态转移表
  - 用状态编码替换状态名称
  - 设定状态变量名称为Q1、Q0
  - 把状态表转换成状态转移表，现态和输入在左侧，次态（Q1*Q0*）和输出在右侧。

状态表

Q1Q0	Q1*Q0*/Z	
	X=0	X=1
00	00/0	01/0
01	11/0	01/0
11	00/0	01/1

状态转移表

Q1	Q0	X	Q1*	Q0*	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	0
0	1	1	0	1	0
1	0	0	d	d	d
1	0	1	d	d	d
1	1	0	0	0	0
1	1	1	0	1	1

根据状态转移表，利用布尔代数或卡诺图，推导出次态函数和输出函数。

次态函数/次态方程：

$Q1^*=\overline{Q1}\bullet Q0\bullet\bar{X}$ 最小风险法：化简不考虑无关项

$Q0^*=\overline{Q1}\bullet Q0\bullet\bar{X}+X\bullet(\overline{Q1}\bullet\overline{Y0}+\overline{Q1}\bullet Q0+Q1\bullet\overline{Q0})$

$=\overline{Q1}\bullet Q0+X\bullet\overline{Q1}+X\bullet Q0$

如果将无关项状态编码10引入化简，则：

$Q0^*=\overline{Q1}\bullet Q0+X$ 最小成本法：化简考虑无关项

输出函数/输出方程：$Z=Q1\bullet X\bullet Q0$；

如果考虑无关状态，则$Z=Q1\bullet X$

4次态方程

- 根据次态逻辑函数和选择的状态记忆单元（触发器），推导出激励函数。
  - 假设采用D触发器，其特征方程Q* = D，则根据次态方程和特征方程推导出激励方程。

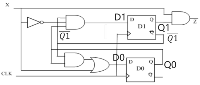
$Q1^*=\overline{Q1}\bullet Q0\bullet\bar{X}$　　$D1=\overline{Q1}\bullet Q0\bullet\bar{X}$

$Q0^*=\overline{Q1}\bullet Q0+X$　　$D0=\overline{Q1}\bullet Q0+X$

  - 输出方程：$Z=Q1\bullet X$

5画出电路

- 根据激励函数和输出函数画出逻辑电路图


X
D1
D
Q
D1
Q1
$\overline{Q1}$
$\overline{Q1}$
Z
D0
D
Q
D0
Q0
CLK

“101”序列检测器电路图

6分析挂起、自启动

- 未用状态分析/挂起/自启动
  - **挂起现象**：未用状态之间形成循环转换，无论如何调整外部输入，电路也无法进入有效工作状态。
  - **自启动**：如果所有未用状态都能在有限个外部输入信号激励下，进入正常工作状态循环，且没有误输出的话。
  - 能够自启动的时序逻辑电路称这种电路是具有“**自启动**”能力的电路。
  - 分析当处于未用状态10时，把Q1=1，Q0=0代入到次态方程和输出方程中检测。
    - 输入x=0，次态是00，输出是0；
    - 输入x=1，次态是01，输出是1；
    - 输出存在错误，需要修正
    - 重新设计逻辑电路图中输出模块

$Q1^*=\overline{Q1}\bullet Q0\bullet\bar{X}$

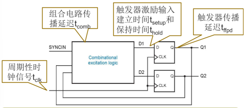
$Q0^*=\overline{Q1}\bullet Q0+X$

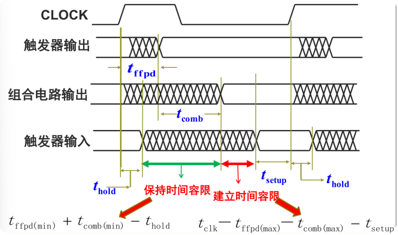
$Z=Q1\bullet X$

$\boxed{Z=Q1\bullet Q0\bullet X}$

7定时分析

- 电路定时分析
  - 电路的工作频率和组合电路传输延迟、触发器建立和保持时间、触发器传输延迟时间密切相关。

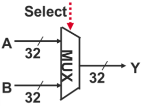

组合电路传播延迟t_{comb}
触发器激励输入建立时间t_{setup}和保持时间t_{hold}
触发器传播延迟t_{ffpd}
周期性时钟信号t_{clk}
SYNCIN
Combinational excitation logic
D1
D2
D
Q
CLK
Q1
Q2


CLOCK
触发器输出
t_{ffpd}
组合电路输出
t_{comb}
触发器输入
t_{hold}
保持时间容限
建立时间容限
t_{setup}
t_{hold}
$t_{ffpd(min)}+t_{comb(min)}-t_{hold}$
$t_{clk}-t_{ffpd(max)}-t_{comb(max)}-t_{setup}$

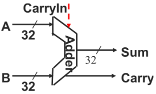
- **定时容限**timing margin：“worse than worst case”
  - 表示电路中的各个部件在不引起电路工作失效的情况下，“比最坏的情况还要坏”多少。

- 建立时间容限$t_{slack}=t_{clk}-t_{ffpd(max)}-t_{comb(max)}-t_{setup}$，$>0$
- 保持时间容限$=t_{ffpd(min)}+t_{comb(min)}-t_{hold}$，$>0$

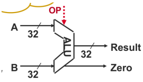
- 经典时序逻辑
  - 计数器
    - 带模
    - 可逆
  - 寄存器
    - 移位寄存器
      - 环形
  - 序列信号实例
- 中央处理器
  - 冯诺依曼结构模型机
  - 两大基本组成部分
    - 数据通路
      - “执行部件”
        - 组合逻辑
          - MUX多路选择

            
            Select
A
32
B
32
MUX
32
Y

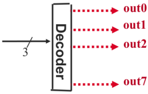
            - 加法器

              
              CarryIn
A
32
Adder
32
Sum
B
32
Carry

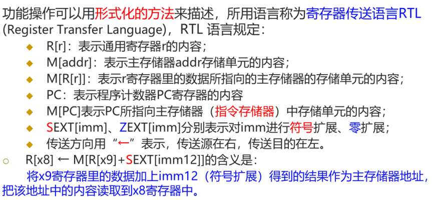
              - 算术逻辑ALU

                
                OP
A
32
ALU
32
Result
B
32
Zero

                - 译码器

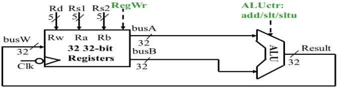
                
                3
Decoder
out0
out1
out2
out7

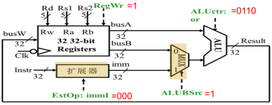
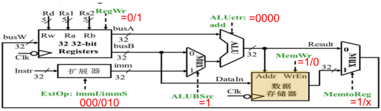
        - 储存元件
          - 输入端状态由时钟决定何时被写入，输出端状态随时可以读出。
        - 只有状态元件能存储信息，所有操作元件都须从状态单元接收输入，并将 输出写入状态单元中。
        - 为使触发器正常工作，必须保证时钟周期不能小于触发器锁存延 迟加 次态信号经过激励逻辑延迟加 触发器的建立时间
        - 触发器锁存延迟加 次态信号经过激励逻辑延迟不能小于触发器的保 持时间
      - 总线/分散
    - 控制器
      - 控制数据通路

- CPU：自动执行程序
  - 取指IFU
    - 取指令，PC+
  - 译码IDU
    - 指令译码，（主存）地址运算
    - 取操作数
  - 执行
    - 计算
      - ALU
    - 存结果
      - 数据存储器
      - 指令存储器
  - 程序-指令序列+数据
    - 指令存储器
    - 数据存储器
  - 异常与中断
    - 异常在CPU内部发生
    - 中断由外部事件引起
- 

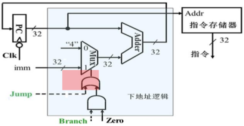
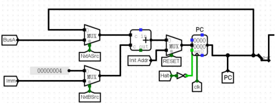
  功能操作可以用形式化的方法来描述，所用语言称为寄存器传送语言RTL（Register Transfer Language），RTL 语言规定：
  - R[r]：表示通用寄存器r的内容；
  - M[addr]：表示主存储器addr存储单元的内容；
  - M[R[r]]：表示r寄存器里的数据所指向的主存储器的存储单元的内容；
  - PC：表示程序计数器PC寄存器的内容
  - M[PC]表示PC所指向主存储器（指令存储器）中存储单元的内容；
  - SEXT[imm]、ZEXT[imm]分别表示对imm进行符号扩展、零扩展；
  - 传送方向用“←”表示，传送源在右，传送目的在左。

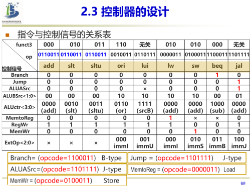
  ○ R[x8] ← M[R[x9]+SEXT[imm12]]的含义是：
  将x9寄存器里的数据加上imm12（符号扩展）得到的结果作为主存储器地址，把该地址中的内容读取到x8寄存器中。
- 立即数扩展器
- ALU

- 单周期CPU
  - 简介：取指令、译码取操作数、计算存结果
    - 指令存储器按照PC的方式存指令，所以取指令默认是用PC取相应位置指令，并PC+4，跳转指令类需要特殊改造
    - 所以R/I/U数据通路介绍了取操作数、计算、存结果的数据通路，译码仅是组合逻辑而已
    - L/S数据通路让数据存储器开始干活，为了兼容其他数据通路增加很多多路选择器，核心是存结果方式的改变
    - B、J数据通路改造了PC+4的简单操作，并改造更多线路让ALU兼容的帮助工作
  - R数据通路
    - 任务：寄存器里读取俩数扔到ALU里操作后把结果写回
      - 地址：指令里
      - ALUA、ALUB都是0，单纯的BUSA、BUSB
      - 进了ALU变BUSW
      - 在一个时钟写回寄存器里，RegWr=1
    - 
      Rd
Rs1
Rs2
RegWr
5
5
5
ALUctr:
add/slt/sltu
busA
busW
Rw
Ra
Rb
32 32-bit
Registers
32
busB
32
32
Clk
ALU
Result
32
    - 控制信号：RegWr=1

- I/U数据通路
  - 任务：R，但一个数可能是I/U立即数
    - 到底是不是：ALUB
  - 
Rd
Rs1
Rs2
RegWr =1
5
5
5
busA
32
busB
32
busW
32
Clk
Rw
Ra
Rb
32 32-bit
Registers
Instr
32
扩展器
imm
32
ExtOp: immI =000
MUX
0
1
ALUBSrc =1
ALUctr: =0110
or
ALU
Result
32
    - 输入指令，立即数转换器，把立即数抠出来！
  - I 控制器输出ExtOp=000：immI，ALUctr为0110（or），ALUBSrc=01，RegWr=1。
  - U 控制器输出ExtOp=001：immU，ALUctr取值为1111（srcB）；ALUBSrc=01，RegWr=1。
- L/S数据通路
  - 任务：L把寄存器写入数据存储器 S把数据存储器写入寄存器
    - 增加数据存储器与选择
    - R[rd]← M[R[rs1]+SEXT(imm12)]
      M[R[rs1]+SEXT(imm12)]←R[rs2]
    - 数据存储器要的Address和DataIn，就是绿色和红色
  - Load
    - BUSA是R[rs1]，给ALUBsrc=1让imm出来和BusA相加得到绿色
    - MemWr=0，数据储存器输出要塞到寄存器的数
    - MemtoReg=1，摒弃ALU（绿色）选择数据储存器的数Load进寄存器
  - Store
    - BUSA是R[rs1]，给ALUBsrc=1让imm出来和BusA相加得到绿色
    - BUSB是R[rs2]，不讲武德连出来进入DataIn
    - MemWr=1，数据储存器写入BUSB
    - 完成，但是RegWr=0封死数据别回来
  - 
Rd
Rs1
Rs2
RegWr
=0/1
5
5
5
busA
32
busB
32
busW
32
Clk
Rw
Ra
Rb
32 32-bit
Registers
Instr
32
扩展器
imm
32
ExtOp: immI/immS
000/010
MUX
0
1
ALUBSrc
=1
ALUctr: =0000
add
ALU
32
Result
MemWr
=1/0
DataIn
Addr
WrEn
数据
存储器
Clk
32
MUX
0
1
MemtoReg
=1/x
  - lw的ALUctr是add；立即数为immI，ExtOp为000； ALUBSrc=01；MemWr为0；MemtoReg为1；RegWr为1。
  - sw的ALUctr是add；立即数为immS，ExtOp为010； ALUBSrc=01；MemWr分别为1；RegWr分别为0。
- B/J型数据通路
  - 任务：（在条件下）跳转！也就是改变PC
  - B
    - 如果寄存器里读取俩数相等，PC就加个立即数
      - 相等：Zero信号为1
    - 不加4！加个多路选择，4或者扩展过的立即数（多连一根过去）
    - ALUctr是sub1000，立即数为immB，故ExtOp=011； ALUASrc=1；ALUBSrc=00； MemWr=0；MemtoReg=0；RegWr=0； Branch=1；Jump=0
  - J
    - 寄存器一个数写入PC+4，且PC加个立即数
    - 在B的判断后再加个J的一票确定
    - 大改ALU！增加一种可能性：ALUA=1不用寄存器数用PC，ALUB=01不用寄存器数用4，然后相加
    - 写入！MemWr=0；MemtoReg=0；RegWr=1；

- ALUctr是add1000，立即数为immJ，故ExtOp=100； ALUASrc=1；ALUBSrc=01； MemWr=0；MemtoReg=0；RegWr=1； Branch=0；Jump=1

- I指令F取Unit
  - 
    PC
32
Clk
imm
"4"
0
Mux
1
32
32
Adder
32
Addr
指令存储器
32
指令
Jump
下地址逻辑
Branch
Zero
  - 
    BusA
MUX
NxtASrc
00000004
Imm
MUX
NxtBSrc
Init Addr
MUX
RESET
PC
Halt
clk
PC

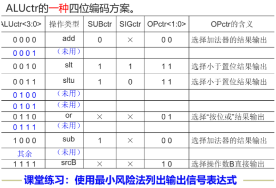
- 控制信号
  - 分析，组合逻辑！
    - 

2.3 控制器的设计

- 指令与控制信号的关系表

funct3 / op / 控制信号	000	010	011	110	无关	010	010	000	无关
op	0110011	0110011	0110011	0010011	0110111	0000011	0100011	1100011	1101111
	add	slt	sltu	ori	lui	lw	sw	beq	jal
Branch	0	0	0	0	0	0	0	1	0
Jump	0	0	0	0	0	0	0	0	1
ALUASrc	0	0	0	0	×	0	0	0	1
ALUBSrc<1:0>	00	00	00	10	10	10	10	00	01
ALUctr<3:0>	0000 (add)	0010 (slt)	0011 (sltu)	0110 (or)	1111 (srcB)	0000 (add)	0000 (add)	1000 (sub)	0000 (add)
MemtoReg	0	0	0	0	0	1	×	×	0
RegWr	1	1	1	1	1	1	0	0	1
MemWr	0	0	0	0	0	0	1	0	0
ExtOp<2:0>	×	×	×	000 immI	001 immU	000 immI	010 immS	011 immB	100 immJ

Branch= (opcode=1100011) B-type
Jump = (opcode=1101111) J-type
ALUASrc=(opcode=1101111) J-type
MemtoReg = (opcode=0000011) Load
MemWr = (opcode=0100011) Store

- 

ALUctr的一种四位编码方案。

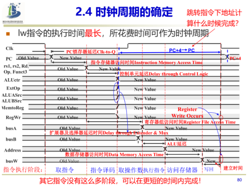
ALUctr<3:0>	操作类型	SUBctr	SIGctr	OPctr<1:0>	OPctr的含义
0000	add	0	×	00	选择加法器的结果输出
0001	（未用）				
0010	slt	1	1	11	选择小于置位结果输出
0011	sltu	1	0	11	选择小于置位结果输出
0100	（未用）				
0101	（未用）				
0110	or	×	×	01	选择"按位或"结果输出
0111	（未用）				
1000	sub	1	×	00	选择加法器的结果输出
其余	（未用）				
1111	srcB	×	×	10	选择操作数B直接输出

课堂练习：使用最小风险法列出输出信号表达式

- 应用
  - 单周期处理器每条指令在一个时钟周期内完成，CPI=1。

    时钟周期的确定：找数据通路中的关键路径

    通常选取最复杂指令所用的指令周期。**R[rd] ← M[R[rs1]+imm]**

    在给出的9条指令中，lw指令周期最复杂。
  - 

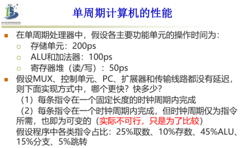
2.4 时钟周期的确定

跳转指令下地址计算什么时候完成?

- lw指令的执行时间最长，所花费时间可作为时钟周期

Clk
PC
Old Value
New Value
PC锁存器延迟Clk-to-Q
PC+4→PC
PC+4
rs1, rs2, Rd, Op, Funct3
指令存储器访问时间Instruction Memory Access Time
控制单元延迟Delay through Control Logic
ALUctr
ExtOp
ALUASrc
ALUBSrc
MemtoReg
RegWr
Register Write Occurs
寄存器组访问时间Register File Access Time
busA
扩展器及选择器延迟时间Delay through Extender & Mux
busB
ALU延迟
Address
数据存储器访问时间Data Memory Access Time
busW
New
建立时间
指令执行阶段:
取指令
指令译码
取操作数
执行指令
访问存储器
写回

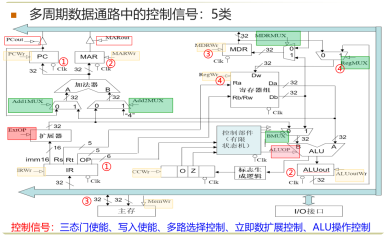
其它指令没有这么多阶段，可以在更短的时间内完成！

- 

  单周期计算机的性能

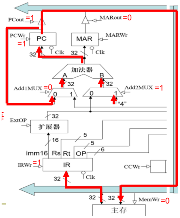
  - 在单周期处理器中，假设各主要功能单元的操作时间为：
    - 存储单元：200ps
    - ALU和加法器：100ps
    - 寄存器堆（读/写）：50ps
  - 假设MUX、控制单元、PC、扩展器和传输线路都没有延迟，则下面实现方式中，哪个更快？快多少？
    （1）每条指令在一个固定长度的时钟周期内完成
    （2）每条指令在一个时钟周期内完成，但时钟周期仅为指令所需，也即为可变的（实际不可行，只是为了比较）
    假设程序中各类指令占比：25%取数、10%存数、45%ALU、15%分支、5%跳转

- 多周期CPU
  - 简介：取指令、译码取操作数、执行（计算存结果）
    - 指令存储器按照PC的方式存指令，所以取指令默认是用PC取相应位置指令，并PC+4，跳转指令类需要特殊改造
    - 所以R/I/U数据通路介绍了取操作数、计算、存结果的数据通路，译码仅是组合逻辑而已
    - L/S数据通路让数据存储器开始干活，为了兼容其他数据通路增加很多多路选择器，核心是存结果方式的改变
    - B、J数据通路改造了PC+4的简单操作，并改造更多线路让ALU兼容的帮助工作
  - 多周期数据通路中的控制信号：5类

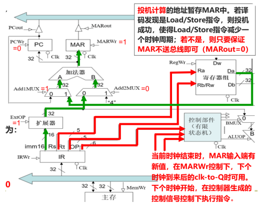
    
    PCout
PCWr
PC
①
MARout
MAR
MARWr
②
Clk
32
加法器
A
B
Add1MUX
Add2MUX
0
1
"4"
ExtOP
扩展器
16
5
6
imm16
Rs
Rt
OP
IRWr
IR
MDRWr
MDRMUX
MDR
③
RegWr
④
Dw
Ra
Da
寄存器组
Rb/Rw
Db
RegMUX
控制部件（有限状态机）
BMUX
ALUOP
ALU
CCWr
O
Z
标志生成逻辑
ALUout
ALUoutWr
MemWr
主存
I/O接口

    控制信号：三态门使能、写入使能、多路选择控制、立即数扩展控制、ALU操作控制

  - 取指令阶段
    - 从PC开始，主线把PC传输到主存（数据&指令存储器），放到指令寄存器IR中
    - PC变为PC+4，由于延迟不会影响上一步
    - 工作信号：有效控制信号及其取值如下： ■ R[IR] ← M[PC]：PCout=1、 MARout=0、MemWr=0、 IRWr=1（不是每个时钟都能更新） ■ PC ← PC +4：Add1MUX=0、 Add2MUX=1、PCWr=1（不是每 个时钟都能更新） ■ 其他写使能信号（如MARWr、 CCWr、MDRWr、ALUoutWr、 RegWr）全部为0

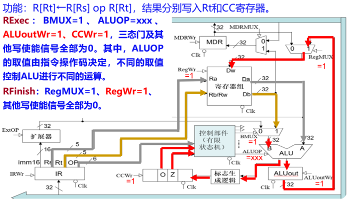
      
      PCout =1
MARout =0
PCWr =1
PC
MAR
MARWr
Clk
32
加法器
A
B
Add1MUX =0
Add2MUX =1
0
1
"4"
ExtOP
扩展器
16
5
6
imm16
Rs
Rt
OP
IRWr =1
IR
CCWr
MemWr =0
主存

  - 译码、取数与扩展
    - IR把指令给控制器生成控制信号，给寄存器译码取数并扩展
    - 为节省L/S指令，投机计算主存地址（寄存器+扩展数）存在MAR（？）
    - 工作信号 有效控制信号取值为： ExtOP=1(符扩)、 Add1MUX=1、 Add2MUX=0、 MARWr=1、 其他写使能信号为0

- 
PCout
MARout
PCWr =0
PC
MAR
MARWr =1
32
Clk
加法器
A
B
Add1MUX =1
Add2MUX =0
"4"
RegWr
Dw
Ra
Da
寄存器组
Rb/Rw
Db
ExtOP =1
扩展器
为:
控制部件（有限状态机）
BMUX
ALUOP
imm16
Rs
Rt
OP
IRWr
IR
0
MemWr
主存

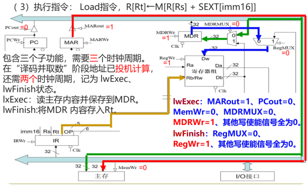
  投机计算的地址暂存MAR中。若译码发现是Load/Store指令，则投机成功，使得Load/Store指令减少一个时钟周期；若不是，则只要保证MAR不送总线即可（MARout=0）

  当前时钟结束时，MAR输入端有新值，在MARWr控制下，下个时钟到来后的clk-to-Q时可用。下个时钟开始，在控制器生成的控制信号控制下执行指令。

- 执行R/I
  - 取出的数ALU运行完，告诉控制器下一周期要写入寄存器
  - 结果早已等在多路选择起，信号过来后写入寄存器
  - 工作信号RExec： BMUX=1 、 ALUOP=xxx 、 ALUoutWr=1、CCWr=1，三态门及其 他写使能信号全部为0。其中，ALUOP 的取值由指令操作码决定，不同的取值 控制ALU进行不同的运算。 RFinish：RegMUX=1、RegWr=1、 其他写使能信号全部为0。
    - 
MDRWr
MDR
32 MDRMUX
RegMUX =1
RegWr =1
Dw
Ra
Da
寄存器组
Rb/Rw
Db
Clk
ExtOP
扩展器
控制部件（有限状态机）
BMUX =1
ALUOP =XXX
ALU
A
B
imm16
Rs
Rt
OP
IRWr
IR
CCWr =1
O
Z
标志生成逻辑
ALUout
ALUoutWr =1

      功能：R[Rt]←R[Rs] op R[Rt]，结果分别写入Rt和CC寄存器。

      RExec： BMUX=1 、 ALUOP=xxx 、 ALUoutWr=1、CCWr=1，三态门及其他写使能信号全部为0。其中，ALUOP的取值由指令操作码决定，不同的取值控制ALU进行不同的运算。

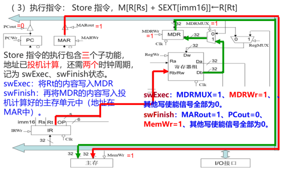
      RFinish：RegMUX=1、RegWr=1、其他写使能信号全部为0。
  - 工作信号IExec：ExtOP=0或1、BMUX=0、 ALUOP=xxx、ALUoutWr=1、CCWr=1， 三态门及其他写使能信号全部为0。ExtOP 和ALUOP取值由指令操作码决定。 IFinish：RegMUX=1、RegWr=1、其他 写使能信号全部为0。 IFinish和RFinish两个状态功能完全一样， 因此，可以将两个状态合并成一个状态： RIFinish。
- 执行L
  - MAR里有地址，主线运输MAR到主存然后把主存的数读出来放到MDR
  - MDR再放到寄存器
  - lwExec：MARout=1、PCout=0、 MemWr=0、MDRMUX=0、 MDRWr=1、其他写使能信号全为0。 lwFinish：RegMUX=0、 RegWr=1、其他写使能信号全为0。
    - 
PCout =0
MARout =1
PCWr
PC
MAR
MARWr
MDRWr =1
MDR
32 MDRMUX =0
RegMUX =0
RegWr =1
Dw
Ra
Da
寄存器组
Rb/Rw
Db
Clk
imm16
Rs
Rt
OP
IRWr
IR
MemWr =0
主存
I/O接口

      （3）执行指令： Load指令，R[Rt]←M[R[Rs] + SEXT[imm16]]

      包含三个子功能，需要三个时钟周期。
      在“译码并取数”阶段地址已投机计算，
      还需两个时钟周期，记为 lwExec、
      lwFinish状态。
      lxExec：读主存内容并保存到MDR。
      lwFinish:将MDR 内容存入Rt。

      lwExec：MARout=1、PCout=0、MemWr=0、MDRMUX=0、MDRWr=1、其他写使能信号全为0。

      lwFinish：RegMUX=0、RegWr=1、其他写使能信号全为0。
- 执行S
  - 把寄存器的数写出放入MDR
  - 通过MAR投机计算的地址，把MDR写入主存
  - 工作信号wExec：MDRMUX=1、MDRWr=1、 其他写使能信号全部为0。 swFinish：MARout=1、PCout=0、 MemWr=1、其他写使能信号全部为0。



（3）执行指令： Store 指令，M[R[Rs] + SEXT[imm16]]←R[Rt]

Store 指令的执行包含三个子功能，地址已投机计算，还需两个时钟周期，记为 swExec、swFinish状态。

swExec：将Rt的内容写入MDR

swFinish：再将MDR的内容写入投机计算好的主存单元中（地址在MAR中）。

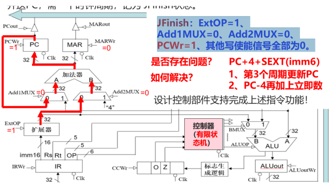
swExec：MDRMUX=1、MDRWr=1、其他写使能信号全部为0。

swFinish：MARout=1、PCout=0、MemWr=1、其他写使能信号全部为0。

PCout =0
MARout =1
PCWr
PC
MAR
MARWr
32
MDRMUX =1
MDRWr =1
MDR
Clk
0
1
RegMUX
RegWr
Dw
Ra
Da
寄存器组
Rb/Rw
Db
imm16
Rs
Rt
OP
5
6
IRWr
IR
MemWr =1
主存
I/O接口

执行J

- 与寄存器，数据存储器无关，把扩展数送给PC，消除之前+4的影响即可
- JFinish：ExtOP=1、 Add1MUX=0、Add2MUX=0、 PCWr=1、其他写使能信号全部为0。



JFinish：ExtOP=1、Add1MUX=0、Add2MUX=0、PCWr=1、其他写使能信号全部为0。

是否存在问题？ PC+4+SEXT(imm6)

如何解决？

1、第3个周期更新PC

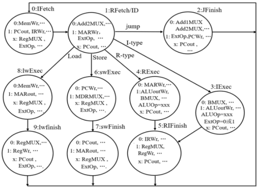
2、PC-4再加上立即数

设计控制部件支持完成上述指令功能！

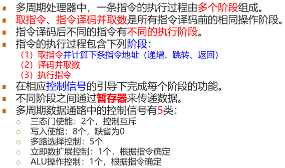
PCout
MARout
PCWr =1
PC
MAR
MARWr =0
32
Clk
加法器
A
B
Add1MUX =0
Add2MUX =0
0
1
"4"
ExtOP =1
扩展器
16
5
6
imm16
Rs
Rt
OP
IRWr
IR
控制器 (有限状态机)
BMUX
ALUOP
ALU
CCWr
O
Z
标志生成逻辑
ALUout
ALUoutWr

控制信号

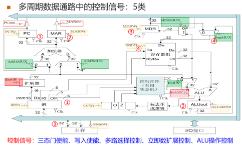
画出状态转换图



0:IFetch
0:MemWr,…
1: PCout, IRWr,…
x: RegMUX ,
ExtOp, …
1:RFetch/ID
0:Add2MUX,…
1: MARWr,
ExtOp, …
x: PCout, …
jump
2:JFinish
0: Add1MUX
Add2MUX,…
1: ExtOp,PCWr, …
x: PCout ,
…
I-type
Load
Store
R-type
8:lwExec
0:MemWr,…
1: MARout, …
x: RegMUX ,
ExtOp, …,
6:swExec
0: PCWr,…
1: MDRMUX,…
x: RegMUX ,
ExtOp, ….
4:RExec
0: MARWr,…
1:ALUoutWr,
BMUX, …
ALUOp=xxx
x: PCout, …
3:IExec
0: BMUX, …
1: ALUoutWr, …
ALUOp=xxx
ExtOp=0或1
x: PCout, …
9:lwfinish
0: RegMUX,…
1: RegWr, …
x: PCout ,
ExtOp, …,
7:swFinish
0: PCout, …
1: MARout, …
x: RegMUX ,
ExtOp, ….
5:RIFinish
0: IRWr, …
1: RegMUX,
RegWr, …
x: PCout, …



- 多周期处理器中，一条指令的执行过程由多个阶段组成。
- 取指令、指令译码并取数是所有指令译码前的相同操作阶段。
- 指令译码后不同的指令有不同的执行阶段。
- 指令的执行过程包含下列阶段：
  （1）取指令并计算下条指令地址（递增、跳转、返回）
  （2）译码并取数
  （3）执行指令
- 在相应控制信号的引导下完成每个阶段的功能。
- 不同阶段之间通过**暂存器**来传递数据。
- 多周期数据通路中的控制信号有5类：
  - 三态门使能：2个，控制互斥
  - 写入使能：8个，缺省为0
  - 多路选择控制：5个
  - 立即数扩展控制：1个，根据指令确定
  - ALU操作控制：1个，根据指令确定



- 多周期数据通路中的控制信号：5类

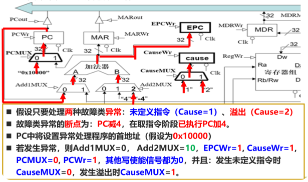
控制信号：三态门使能、写入使能、多路选择控制、立即数扩展控制、ALU操作控制

PCout
MARout
PCWr
PC
①
MAR
MARWr
②
MDRMUX
MDRWr
③
MDR
RegMUX
④
RegWr
④
Dw
Ra
Da
寄存器组
Rb/Rw
Db
加法器
Add1MUX
Add2MUX
"4"
ExtOP
扩展器
控制部件（有限状态机）
BMUX
ALUOP
ALU
imm16
Rs
Rt
OP
IRWr
IR
①
CCWr
O
Z
标志生成逻辑
②
ALUout
ALUoutWr
③
MemWr
主存
I/O接口

- 微程序控制器

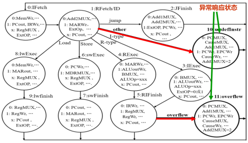
异常处理

- 带异常处理的数据通路

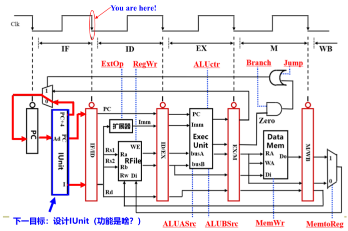

PCout
MARout
MDRM
EPCWr
EPC
MDRWr
MDR
PCWr
PC
MAR
MARWr
32
Clk
CauseWr
cause
RegWr
PCMUX
Dw
Ra
寄存器组
Rb/Rw
加法器
A
B
CauseMUX
"0x10000"
Add1MUX
Add2MUX
"1"
"2"
"4"
"-4"

■ 假设只要处理两种故障类异常：未定义指令（Cause=1）、溢出（Cause=2）
■ 故障类异常的断点为：PC减4，在取指令阶段已执行PC加4。
■ PC中将设置异常处理程序的首地址（假设为0x10000）
■ 若发生异常，则Add1MUX=0， Add2MUX=10，EPCWr=1, CauseWr=1, PCMUX=0, PCWr=1，其他写使能信号都为0，并且：发生未定义指令时CauseMUX=0，发生溢出时CauseMUX=1。

-
  - 加入异常处理后的有限状态转换图

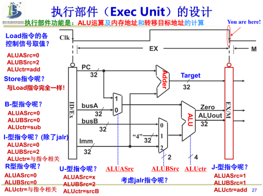

0:IFetch
1:RFetch/ID
2:JFinish
异常响应状态
jump
other
I-type
R-type
Load
Store
10:undefInstr
8:lwExec
6:swExec
4:RExec
3:IExec
9:lwfinish
7:swFinish
5:RIFinish
11:overflow
overflow

- 流水线CPU与冒险
  - 流水段分析
    - 简介：取指令、寄存器读、执行、（读数据存储），写回寄存/PC
    - 所有指令都是这些模块如何做到/不出问题
  - 流水数据通路与控制
    - 数据通路
      -


You are here!
Clk
IF
ID
EX
M
WB
ExtOp
RegWr
ALUctr
Branch
Jump
PC
PC+4
Ad
IUnit
I
IF/ID
扩展器
Imm
Rs1
Rs2
Rd
WE
Ra
Rb
Rw
Di
RFile
ID/EX
Exec Unit
busA
busB
EX/M
Zero
Data Mem
RA
WA
Do
M/WB
ALUASrc
ALUBSrc
MemWr
MemtoReg

下一目标：设计IUnit（功能是啥？）

      - IUnit
        - 先+4，多路选择器用控制信号决定用+4的结果还是后面ALU算的新PC
      - ExecUnit
        - 全部兼容后的单周期寄存器-数据存储前
          -



执行部件（Exec Unit）的设计

执行部件功能是：ALU运算及内存地址和转移目标地址的计算

Load指令的各控制信号取值？
ALUASrc=0
ALUBSrc=2
ALUctr=add

Store指令呢？
与Load指令完全一样！

B-型指令呢？
ALUASrc=0
ALUBSrc=0
ALUctr=sub

I-型指令呢？(除了jalr)
ALUASrc=0
ALUBSrc=2
ALUctr=与指令相关

R型指令呢？
ALUASrc=0
ALUBSrc=0
ALUctr=与指令相关

U-型指令呢？
ALUASrc=x
ALUBSrc=2
ALUctr=srcB

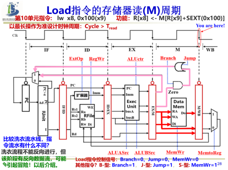
考虑jalr指令呢？

J-型指令呢？
ALUASrc=1
ALUBSrc=1
ALUctr=add

27

You are here!
Clk
EX
M
PC
32
Adder
Target
ID/Ex
busA
busB
Imm
"4"
1
0
2
ALU
Zero
ALUout
EX/M
4
ALUASrc
ALUBSrc
ALUctr

      - 各异三件事：数据存储读出，写回存储器，PC修改，与单周期没有本质区别



Load指令的存储器读(M)周期

第10单元指令： lw x8, 0x100(x9)　功能：R[x8] <- M[R[x9]+SEXT(0x100)]

以最长操作为准设计时钟周期：Cycle > T_{read}

You are here!

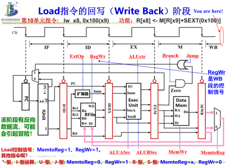
Clk
IF
ID
EX
M
WB
ExtOp
RegWr
ALUctr
Branch
Jump
PC
扩展器
Imm
Rs1
Rs2
Rd
WE
Ra
Rb
Rw
Di
RFile
IUnit
PC+4
Ad
I
IF/ID
ID/EX
EX/M
M/WB
Exec Unit
busA
busB
Zero
Data Mem
RA
WA
Do
1
0
ALUASrc
ALUBSrc
MemWr
MemtoReg

比较洗衣流水线，指令流水有什么不同？

洗衣流程不能反向进行，但该阶段有反向数据流，可能令引起冒险！以后介绍。

Load指令控制信号：Branch=0、Jump=0、MemWr=0

其他指令？B-型: Branch=1， J-型: Jump=1， S-型: MemWr=1



Load指令的回写（Write Back）阶段

You are here!

第10单元指令： lw x8, 0x100(x9)　功能：R[x8] <- M[R[x9]+SEXT(0x100)]

Clk
IF
ID
EX
M
WB
ExtOp
RegWr
ALUctr
Branch
Jump
PC
扩展器
Imm
Rs1
Rs2
Rd
WE
Ra
Rb
Rw
Di
RFile
IUnit
PC+4
Ad
I
IF/ID
ID/EX
EX/M
M/WB
Exec Unit
busA
busB
Zero
Data Mem
RA
WA
Do
1
0
ALUASrc
ALUBSrc
MemWr
MemtoReg

RegWr是WB段的控制信号

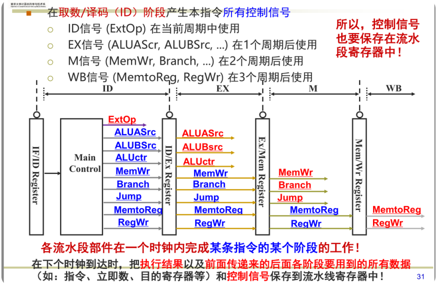
该阶段有反向数据流，可能会引起冒险！

Load控制信号：MemtoReg=1、RegWr=1。

其他指令呢？

R-型、I-型运算、U-型、J-型: MemtoReg=0、RegWr=1， B-型、S-型: MemtoReg=x、RegWr=0

- 控制器



- 在取数/译码（ID）阶段产生本指令所有控制信号
  - ID信号 (ExtOp) 在当前周期中使用
  - EX信号 (ALUAScr, ALUBSrc, ...) 在1个周期后使用
  - M信号 (MemWr, Branch, ...) 在2个周期后使用
  - WB信号 (MemtoReg, RegWr) 在3个周期后使用

所以，控制信号也要保存在流水段寄存器中！

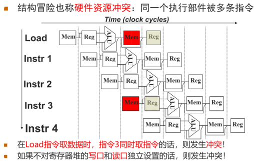
ID
EX
M
WB
IF/ID Register
Main Control
ExtOp
ALUASrc
ALUBSrc
ALUctr
MemWr
Branch
Jump
MemtoReg
RegWr
ID/Ex Register
ALUASrc
ALUBsrc
ALUctr
MemWr
Branch
Jump
MemtoReg
RegWr
Ex/Mem Register
MemWr
Branch
Jump
MemtoReg
RegWr
Mem/Wr Register
MemtoReg
RegWr

各流水段部件在一个时钟内完成某条指令的某个阶段的工作！

在下个时钟到达时，把执行结果以及前面传递来的后面各阶段要用到的所有数据（如：指令、立即数、目的寄存器等）和控制信号保存到流水线寄存器中！

- 一次性产生所有信号，不用的直接传给下个信号

- 流水冒险解决与应用分析
  - 分三种：结构冒险、数据冒险、控制冒险。
    - 结构冒险：同一个部件同时被不同指令所用
      - 结构冒险解决方法：将寄存器读口和写口独立开来，相当于两个部件。

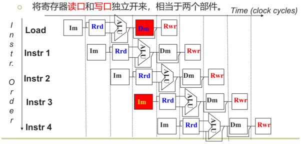


- 结构冒险也称硬件资源冲突：同一个执行部件被多条指令使用

Time (clock cycles)
Load
Instr 1
Instr 2
Instr 3
Instr 4
Mem
Reg
ALU
Mem
Reg

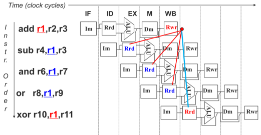
- 在Load指令取数据时，指令3同时取指令的话，则发生冲突！
- 如果不对寄存器堆的写口和读口独立设置的话，则发生冲突！

- 解决方法：
  - 为了避免结构冒险，规定流水线数据通路中功能部件的设置原则为：每个部件只在特定的流水段中使用！（如：ALU总在第三阶段被用！）
  - 将Instruction Memory (Im) 和 Data Memory (Dm)分开
  - 将寄存器读口和写口独立开来，相当于两个部件。


Time (clock cycles)
Instr. Order
Load
Instr 1
Instr 2
Instr 3
Instr 4
Im
Rrd
ALU
Dm
Rwr

- 数据冒险：后面指令用到前面指令结果时，前面指令的结果 还没产生。
  - 数据冒险：基本流水线的写后读RAW冒险
    - 相关的数据是ALU结果：可以通过转发解决
    - 相关的数据是DM读出的内容：随后的指令需被阻塞一个时钟

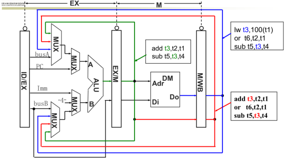
- 举例：以下指令序列中，寄存器r1会发生数据冒险


Time (clock cycles)
IF
ID
EX
M
WB
Instr. Order
add r1,r2,r3
sub r4,r1,r3
and r6,r1,r7
or r8,r1,r9
xor r10,r1,r11
Im
Rrd
ALU
Dm
Rwr

最后一条指令的r1才是新的值！

如何解决这个问题？

硬件上的改动以支持“转发”技术


EX
M
ID/EX
EX/M
M/WB
MUX
busA
PC
Imm
busB
“4”
A
B
ALU
DM
Adr
Di
Do

lw t3,100(t1)
or t6,t2,t1
sub t5,t3,t4

add t3,t2,t1
sub t5,t3,t4

add t3,t2,t1
or t6,t2,t1
sub t5,t3,t4

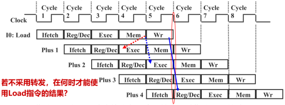
如果指令序列为

lw t3, 100(t1)
or t6, t3, t1
sub t5, t3, t4

能用“转发”技术解决第1、2两条指令间的数据冒险吗？

请看后面的幻灯片！

- Load指令引起的延迟现象


Cycle 1
Cycle 2
Cycle 3
Cycle 4
Cycle 5
Cycle 6
Cycle 7
Cycle 8
Clock
I0: Load
Plus 1
Plus 2
Plus 3
Plus 4
Ifetch
Reg/Dec
Exec
Mem
Wr

若不采用转发，在何时才能使用Load指令的结果?

- Load指令最早在哪个流水线寄存器中开始有后续指令需要的值?

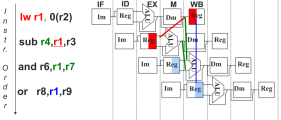
实际上，在第四周期结束时，数据在流水段寄存器中已经有值。

采用数据转发技术可以使load指令后面第二条指令得到所需的值

但不能解决load指令和随后第一条指令间的数据冒险，要延迟执行一条指令！

这种load指令和随后指令间的数据冒险，称为“装入- 使用数据冒险 (load- use Data Hazard)”

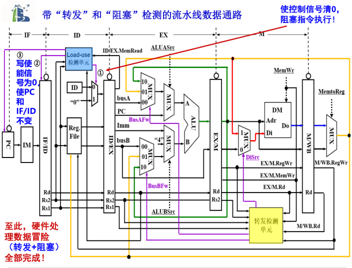
- 转发Forwarding技术使Load-use冒险只需延迟一个周期


Time (clock cycles)
IF
ID
EX
M
WB
Instr. Order
lw r1, 0(r2)
sub r4,r1,r3
and r6,r1,r7
or r8,r1,r9
Im
Reg
ALU
Dm
Reg

采用“转发”后仅第二条指令 SUB r7,r1,r3 不能按时执行！

发生“装入- 使用数据冒险”时，需要对load后的指令阻塞一个时钟周期！

数据冒险处理最佳方案：“转发” ＋ “Load-use阻塞”

- 

带“转发”和“阻塞”检测的流水线数据通路

使控制信号清0，阻塞指令执行！

③ 写使能信号为0，使PC和IF/ID不变

至此，硬件处理数据冒险（转发+阻塞）全部完成！

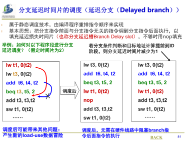
IF
ID
EX
M
Load-use 检测单元
ID/EX.MemRead
ALUASrc
②
①
PC
IM
IF/ID
ID
“0”
Reg. File
ID/EX
busA
PC
Imm
busB
BusAFw
BusBFw
“4”
MUX
ALU
A
B
ALUBSrc
EX/M
MemWr
DM
Adr
Do
Di
DiSrc
EX/M.RegWr
EX/M.MemWr
EX/M.Rd
M/WB
MemtoReg
M/WB.RegWr
M/WB.Rd
Rd
Rs2
Rs1
转发检测单元

- 控制冒险：转移或异常改变执行流程，后继指令在目标地址 产生前已被取出。

控制冒险

- 基本做法
  - 总预测条件不满足(not taken)，即：继续执行分支指令的后续指令
    可加启发式规则：
    在特定情况下总是预测满足(taken)，其他情况总是预测不满足
  - 预测失败时，需把流水线中三条错误预测指令丢弃掉
    - 将被丢弃指令的控制信号值或指令设置为0
      （注：涉及当时在IF、ID和EX三个阶段的指令）
- 性能
  - 如果转移概率是50%，则预测正确率仅有50%
- 预测错误的代价
  - 预测错误的代价与何时能确定是否转移有关。越早确定代价越少
  - 可否把“是否转移”的确定工作提前，而不等到MEM阶段才确定？

- 缩短分支延迟，减少错误预测代价
  - 可以将 “转移地址计算”和“分支条件判断”操作调整到ID阶段来缩短延迟
    - 将转移地址生成从M阶段移到ID阶段，可以吗？为什么？
      （可：IF/ID流水段寄存器中已有PC的值，ID段已有扩展后立即数Imm）
    - 将“判0”操作从EX阶段移到ID阶段，可以吗？为什么？
      (用逻辑运算(如，先按位异或，再结果各位相或)直接比较Rs1和Rs2的值)
      (简单判断用逻辑运算，复杂判断可以用专门指令生成条件码)
      (许多条件判断都很简单)
- 预测错误的检测和处理（称为“冲刷、冲洗” -- Flush）
  - 当Branch=1并且Zero=1时，发生转移（taken）
  - 增加控制信号：IF.Flush=Branch && Zero，取值为1时，说明预测失败
  - 预测失败 (条件满足) 时，完成以下两件事（延迟损失时间片C=1时）：
    ① 将转移目标地址->PC
    ② 清除IF段中取出的指令，即：将IF/ID中的指令字清0，转变为nop指令

- 

分支延迟时间片的调度（延迟分支（**Delayed branch**））

- 属于静态调度技术，由编译程序重排指令顺序来实现
- 基本思想：把分支指令前面与分支指令无关的指令调到分支指令后面执行，以填充延迟损失时间片（也称分支延迟槽Branch Delay slot），不够时用nop填充

举例：如何对以下程序段进行分支延迟调度？（假定时间片为2）

```
lw t1, 0(t2)
lw t3, 0(t2)
add  t6, t4, t2
beq t3, t5, 2
add t3, t3,t2
sw t1, 0(t2)
    ……
```

调度后

```
lw t3, 0(t2)
add  t6, t4, t2
beq t3, t5, 2
lw t1, 0(t2)
nop
add t3, t3,t2
sw t1, 0(t2)
```

若分支条件判断和目标地址计算提前到ID阶段，则分支延迟时间片减少为1

```
lw t3, 0(t2)
add  t6, t4, t2
beq t3, t5, 2
lw t1, 0(t2)
add t3, t3,t2
sw t1, 0(t2)
    ……
```

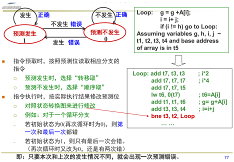
调度后可能带来其他问题：产生新的load-use数据冒险

调度后，无需在硬件线路中阻塞branch指令后面指令的执行

BACK

- 动态预测基本思想：动态预测的准确率可达90%
  - 利用最近转移发生的情况，来预测下一次可能转移还是不转移
  - 根据实际情况来调整预测
  - 转移发生的历史情况记录在BHT中（有多个不同的名称）
    - 分支历史记录表BHT（Branch History Table）
    - 分支预测缓冲BPB（Branch Prediction Buffer）
    - 分支目标缓冲BTB（Branch Target Buffer）
  - 每个表项由分支指令地址低位作索引，故在IF阶段就可以取到预测位
    - 低位地址相同的分支指令共享一个表项，所以，可能取的是其他分支指令的预测位。会不会有问题？
    - 由于仅用于预测，所以不影响执行结果

- 采用一位预测位：总是按上次实际发生的情况来预测下次
  - 1表示最近一次发生过转移（taken），0表示未发生（not taken）
  - 预测时，若为1，则预测下次taken，若为0，则预测下次not taken
  - 实际执行时，若预测错，则该位取反，否则，该位不变
  - 可用一个简单的预测状态图表示
  - 缺点：当连续两次的分支情况发生改变时，预测错误
    - 例如，循环迭代分支时，第一次和最后一次会发生预测错误，因为循环的第一次和最后一次都会改变分支情况，而在循环中间的各次总是会发生分支，按上次的实际情况预测时，都不会错。
- 采用二位预测位
  - 用2位组合四种情况来表示预测和实际转移情况
  - 按照预测状态图进行预测和调整
  - 在连续两次分支发生不同时，只会有一次预测错误


发生 正确
不发生 正确
不发生 错误
预测发生
1
预测不发生
0
发生 错误

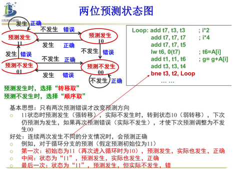
Loop: g = g +A[i];
         i = i+ j;
         if (i != h) go to Loop:
Assuming variables g, h, i, j ~ t1, t2, t3, t4 and base address of array is in t5

```
Loop: add t7, t3, t3      ; i*2
      add t7, t7, t7      ; i*4
      add t7, t7, t5
      lw t6, 0(t7)        ; t6=A[i]
      add t1, t1, t6      ; g= g+A[i]
      add t3, t3, t4      ; i=i+j
      bne t3, t2, Loop
      … …
```

- 指令预取时，按照预测位读取相应分支的指令
  - 预测发生时，选择“转移取”
  - 预测不发生时，选择“顺序取”
- 指令执行时，按实际执行结果修改预测位
  - 对照状态转换图来进行修改
  - 例如：对于一个循环分支
    - 若初始状态为0(再次循环时为0)，则第一次和最后一次都错
    - 若初始状态为1，则只有最后一次会错。（再次循环时又改为0，还是有两次错）

即：只要本次和上次的发生情况不同，就会出现一次预测错误。

77

- 

两位预测状态图

发生 正确
不发生 错误
预测发生
11
预测发生
10
发生 正确
发生 错误
不发生 错误
不发生 正确
预测不发生
01
预测不发生
00
发生 错误
不发生 正确

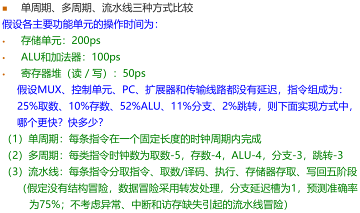
```
Loop: add t7, t3, t3      ; i*2
      add t7, t7, t7      ; i*4
      add t7, t7, t5
      lw t6, 0(t7)        ; t6=A[i]
      add t1, t1, t6      ; g= g+A[i]
      add t3, t3, t4
      bne t3, t2, Loop
      … …
```

预测发生时，选择“转移取”
预测不发生时，选择“顺序取”

基本思想：只有两次预测错误才改变预测方向
- 11状态时预测发生（强转移），实际不发生时，转到状态10（弱转移），下次仍预测为发生，如果再次预测错误（实际不发生），才使下次预测调整为不发生00

好处：连续两次发生不同的分支情况时，会预测正确
- 例如，对于循环分支的预测（假定预测初始位为11）
- 第一次：初始态为11（再次进入循环时为10），预测发生，实际也发生，正确
- 中间：状态为“11”，预测发生，实际也发生，正确
- 最后一次：状态为“11”，预测发生，但实际不发生，错

应用

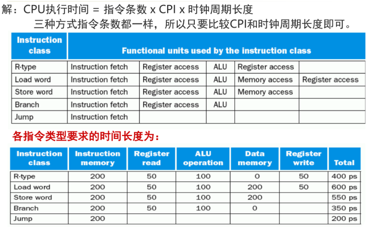
- 

■ 单周期、多周期、流水线三种方式比较

假设各主要功能单元的操作时间为：
- 存储单元：200ps
- ALU和加法器：100ps
- 寄存器堆（读 / 写）：50ps

假设MUX、控制单元、PC、扩展器和传输线路都没有延迟，指令组成为：25%取数、10%存数、52%ALU、11%分支、2%跳转，则下面实现方式中，哪个更快？快多少？

（1）单周期：每条指令在一个固定长度的时钟周期内完成
（2）多周期：每类指令时钟数为取数-5，存数-4，ALU-4，分支-3，跳转-3
（3）流水线：每条指令分取指令、取数/译码、执行、存储器存取、写回五阶段（假定没有结构冒险，数据冒险采用转发处理，分支延迟槽为1，预测准确率为75%；不考虑异常、中断和访存缺失引起的流水线冒险）

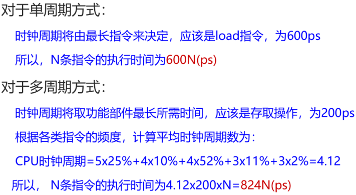
- 

解：CPU执行时间 = 指令条数 x CPI x 时钟周期长度
三种方式指令条数都一样，所以只要比较CPI和时钟周期长度即可。

Instruction class	Functional units used by the instruction class				
R-type	Instruction fetch	Register access	ALU	Register access	
Load word	Instruction fetch	Register access	ALU	Memory access	Register access
Store word	Instruction fetch	Register access	ALU	Memory access	
Branch	Instruction fetch	Register access	ALU		
Jump	Instruction fetch				

各指令类型要求的时间长度为：

Instruction class	Instruction memory	Register read	ALU operation	Data memory	Register write	Total
R-type	200	50	100	0	50	400 ps
Load word	200	50	100	200	50	600 ps
Store word	200	50	100	200		550 ps
Branch	200	50	100	0		350 ps
Jump	200					200 ps

- 

对于单周期方式：

时钟周期将由最长指令来决定，应该是load指令，为600ps

所以，N条指令的执行时间为600N(ps)

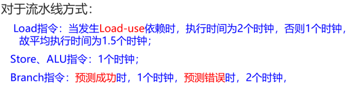
对于多周期方式：

时钟周期将取功能部件最长所需时间，应该是存取操作，为200ps

根据各类指令的频度，计算平均时钟周期数为：

CPU时钟周期=5x25%+4x10%+4x52%+3x11%+3x2%=4.12

所以， N条指令的执行时间为4.12x200xN=824N(ps)

- 

对于流水线方式：

Load指令：当发生Load-use依赖时，执行时间为2个时钟，否则1个时钟，故平均执行时间为1.5个时钟；

Store、ALU指令：1个时钟；

Branch指令：预测成功时，1个时钟，预测错误时，2个时钟，

所以：平均约为：0.75x1+0.25x2=1.25个；

Jump指令：2个时钟（总要等到译码阶段结束才能得到转移地址）

平均CPI为：1.5x25%+1x10%+1x52%+1.25x11%+2x2%=1.17

所以， N条指令的执行时间为1.17x200xN=234N(ps)

综上所述，流水线方式的执行速度最快，
与单周期相比，约为600ps/234ps=2.56倍；
与多周期相比，约为824ps/234ps=3.52倍；

- **微积分**